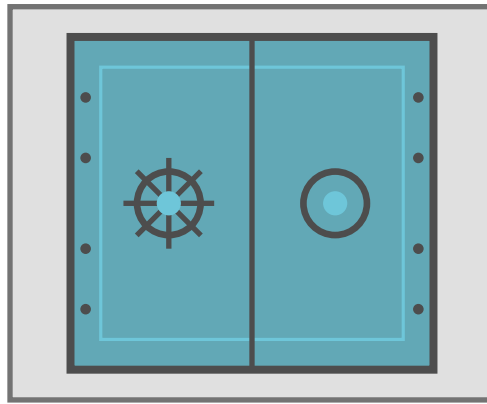


Criptografía y Seguridad

Criptografía: Protocolos y otros Protocolos, PKI y Secreto Compartido



Apunte integral — Teoría + Práctica

Clase 5 · Capítulo 11, *Computer Security: Art and Science* (M. Bishop) · RFC 8446
(TLS 1.3)

Instituto Tecnológico de Buenos Aires (ITBA)

Documento elaborado a partir de las diapositivas oficiales de la teórica y de la práctica, y de las transcripciones de ambas clases.

Índice

1. Introducción: del cifrado simétrico a los protocolos	2
2. Distribución simétrica de claves: el KDC	2
3. Protocolo Needham–Schroeder (clave simétrica)	3
3.1. Primera aproximación (ingenua)	4
3.2. Segunda aproximación (con nonces)	4
3.3. Ataque residual y modificación Denning–Sacco	5
4. El problema de las claves públicas: Man-in-the-Middle	5
5. Infraestructura de Clave Pública (PKI)	7
5.1. Certificados	7
5.2. Cadenas de firmas y CA raíces	7
5.3. Certificados X.509	8
5.4. Revocación: CRL y OCSP	9
6. Canal seguro: SSL / TLS	10
6.1. TLS Record	10
6.2. Tipos de mensajes y material criptográfico	10
6.3. TLS Handshake (diagrama de secuencia)	11
7. Criptografía de umbrales (Threshold Cryptography)	12
8. Secreto compartido de Shamir	13
8.1. Intuición geométrica	13
8.2. Ejemplo de cifrado (reparto) — esquema (3, 5)	14
8.3. Ejemplo de descifrado (reconstrucción por Lagrange)	15
9. Cross-check teoría vs. práctica	16
Lectura recomendada	16

1 Introducción: del cifrado simétrico a los protocolos

Esta clase cierra el bloque de criptografía con los **protocolos** que hacen utilizable todo lo visto: cómo dos partes acuerdan una clave, cómo se asocia una clave pública a una identidad (PKI), cómo se construye un canal seguro (TLS) y cómo se reparte un secreto entre varios custodios (Shamir). La idea central es que los *algoritmos* criptográficos son piezas; los *protocolos* son las reglas de intercambio de mensajes que los combinan para lograr propiedades de seguridad frente a un adversario.

Punto de partida (repaso)

Un criptosistema **CCA-Secure** permite enviar información manteniendo **confidencialidad** e **integridad**:

$$A \xrightarrow{\text{Enc}_k(M)} B$$

Pero requiere que ambas partes conozcan una misma clave k . De ahí surge el problema que organiza toda la clase: *¿cómo logran A y B compartir esa clave?*

Intercambio de claves como protocolo $\Pi(n)$

Un **intercambio de claves** es un protocolo $\Pi(n)$ ejecutado por dos partes:

$$\Pi : (n) \rightarrow (\text{Tran}, k_a, k_b)$$

- No tiene entrada salvo el **parámetro de seguridad n** .
- La salida es: un conjunto de **mensajes intercambiados** (Tran), una clave k_a conocida solo por una parte y una clave k_b conocida solo por la otra.

Condición fundamental: $k_a = k_b$ (ambas partes terminan con la misma clave).

Aporte de la práctica — limitaciones de la clave privada (KDC)

La práctica encuadra toda la clase en las **tres limitaciones de la criptografía de clave privada (simétrica)**:

1. **Distribución de la clave** $\rightarrow$ se ataca con un **KDC** (Key Distribution Center).
2. **Almacenamiento de la clave** $\rightarrow$ también lo alivia el KDC (cada uno guarda una sola clave).
3. **Sistemas abiertos** (*open systems*) $\rightarrow$ motiva la *revolución de clave pública* (PKI).

Con n participantes que quieren hablar de a pares, un esquema todos-con-todos requiere $\binom{n}{2}$ claves; el KDC lo reduce a n claves (una por participante con el centro).

2 Distribución simétrica de claves: el KDC

KDC — Key Distribution Center

Un **punto único** en el que se concentran las relaciones de confianza. Cada entidad U comparte una clave k_U *solo con el KDC* (k_A para A , k_B para B , etc.). Cuando A quiere hablar con B :

1. $A \rightarrow \text{KDC}$: “quiero comunicarme con B ”.

2. El KDC genera una **clave de sesión** s y la entrega cifrada a cada uno: $KDC \rightarrow A : \{s\} k_A$ y $KDC \rightarrow B : \{s\} k_B$.
3. A y B se comunican entre sí cifrando con s : $\{\dots\} s$.

La clave de sesión s se genera *ad hoc* para esa conversación.

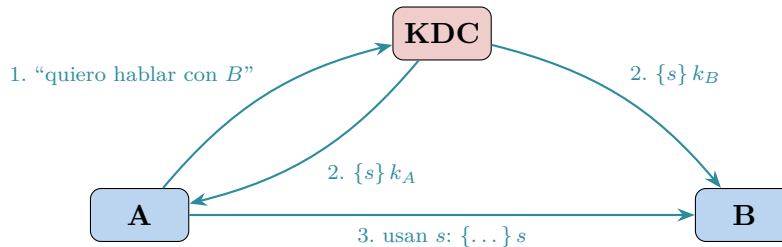


Figura 1: Esquema del KDC (recreación de la diapositiva de práctica). Ejemplo de uso real: **Kerberos** y **Active Directory**.

Ventajas	Cada participante guarda <i>solo</i> k_U . Hay un único sitio sensible: el KDC.
Desventajas	Centralización: si el KDC es atacado o falla, <i>se cae todo</i> . Es el único punto de confianza (y de falla).

Aclaración del profe — KDC vs. PKI

El KDC y la PKI *ambos* requieren confiar en un tercero, pero de naturaleza distinta:

- En el **KDC** (mundo simétrico) confío en un servicio *en línea* que *conoce todas las claves* y participa en cada conversación.
- En la **PKI** (mundo asimétrico) confío en una **autoridad certificante** que solo *firma* la asociación clave-identidad; *no* conoce las claves privadas ni participa en cada conexión.

La PKI *no es aplicable* a los criptosistemas simétricos: resuelve la integridad/origen de *claves públicas*.

3 Protocolo Needham-Schroeder (clave simétrica)

Es el protocolo de intercambio de claves simétricas que usan **Kerberos** y **Active Directory** (con modificaciones). Requiere un servicio centralizado (KDC) y **genera claves de sesión entre pares**. *Hipótesis:* cada entidad comparte una clave con el KDC. A lo largo de las sucesivas “aproximaciones” se ven los ataques y sus arreglos.

Notación. Se escribe $e_k(M) = \{M\} k$ (cifrado de M bajo k), y $\parallel$ denota concatenación.

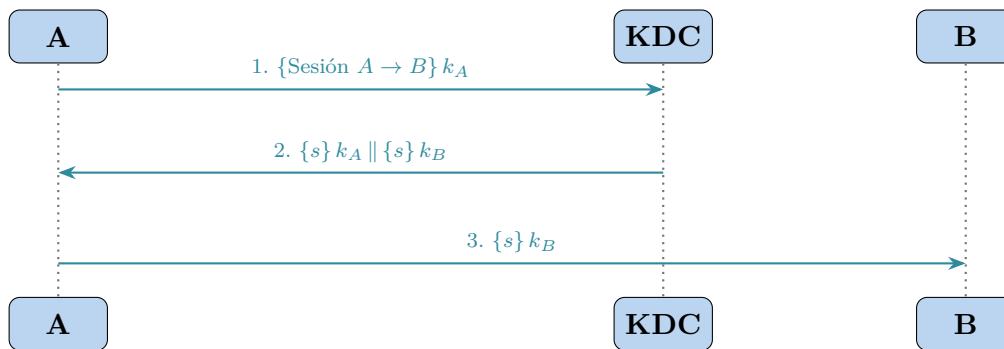


Figura 2: Primera aproximación: A pide la sesión, el KDC le devuelve s cifrada para ambos, A reenvía a B su copia. (Recreación de la diapositiva.)

3.1 Primera aproximación (ingenua)

Problemas de la primera aproximación

- **Repetición (replay):** un atacante graba los mensajes de A hacia B y luego reenvía $\{s\} k_B$ y los siguientes. B no tiene forma de saber que no está hablando con A .
- **Reúso de claves (key reuse):** un atacante graba el mensaje del KDC a A y se lo reenvía cuando A inicia otra conversación. A y B terminan usando la *misma* clave de sesión vieja.

3.2 Segunda aproximación (con nonces)

Se agregan **nonces** (números aleatorios de un solo uso) r_1, r_2 para garantizar **frescura** (anti-replay) y autenticación mutua.

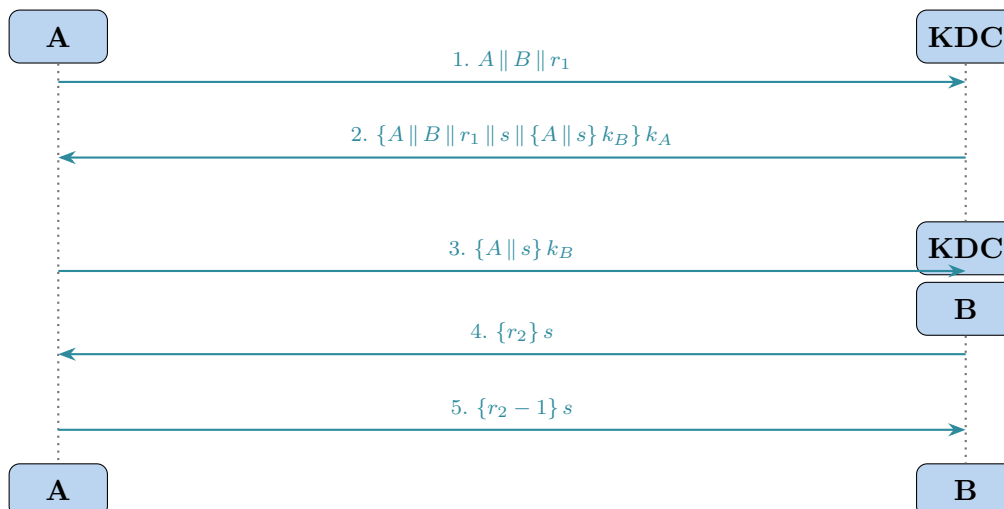


Figura 3: Segunda aproximación con nonces. (Recreación de la diapositiva.) El mensaje 2 está cifrado con k_A (clave A–KDC), el mensaje 3 contiene el “ticket” $\{A || s\} k_B$ que solo B abre.

Por qué funciona — explicación mensaje a mensaje

Segundo mensaje

Cifrado con la clave compartida A–KDC $\Rightarrow$ *proviene del KDC*. No es repetición

porque el r_1 recibido coincide con el enviado.

Cuarto mensaje

Solo B puede enviarlo (solo B obtiene s del ticket). Avisa a A de un intento de comunicación enviando $\{r_2\} s$.

Quinto mensaje

A confirma la comunicación devolviendo $\{r_2 - 1\} s$. B sabe que *no es una repetición* (por r_2): prueba que del otro lado hay alguien que conoce s *ahora*.

3.3 Ataque residual y modificación Denning–Sacco

Problema: clave de sesión vieja comprometida

Escenario: el atacante E obtiene una **clave de sesión antigua** s . Comienza directamente por el **tercer paso**, reenviando el ticket viejo $\{A \parallel s\} k_B$:

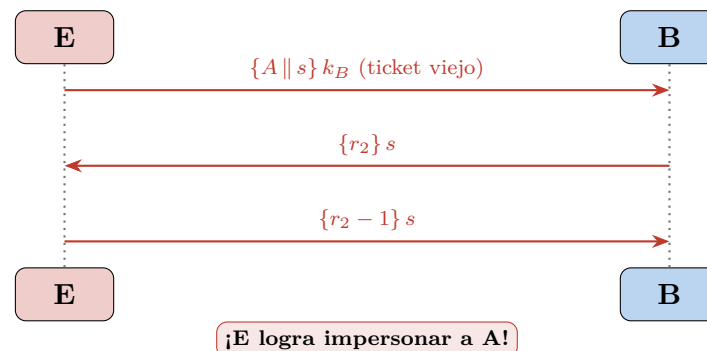


Figura 4: Ataque por reúso de clave de sesión vieja. (Recreación de la diapositiva.)

Modificación Denning–Sacco — agregar timestamps

El arreglo propuesto es incluir un **timestamp** T dentro del ticket. El ticket pasa a ser $\{A \parallel T \parallel s\} k_B$: B verifica que T sea *reciente* y rechaza tickets viejos, neutralizando el ataque anterior.

Aclaración del profe — nonces, frescura y anti-replay

Los ataques de **replay** (repetición de mensajes) y de **key reuse** aparecen en *cualquier* protocolo (KDC, Diffie–Hellman, etc.). Las dos herramientas para garantizar **frescura** son:

- **Nonces** (r_1, r_2): números aleatorios de un solo uso; quien responde con el nonce correcto demuestra que participa *ahora*.
- **Timestamps** (T): marcas de tiempo que prueban que el mensaje es reciente y no una grabación vieja.

4 El problema de las claves públicas: Man-in-the-Middle

Con criptografía asimétrica desaparece la necesidad de compartir un secreto previo, pero **aparece otro problema**: ¿de dónde se obtienen las claves públicas? Se confía *implícitamente* en la asociación entre una clave y la identidad de su dueño.

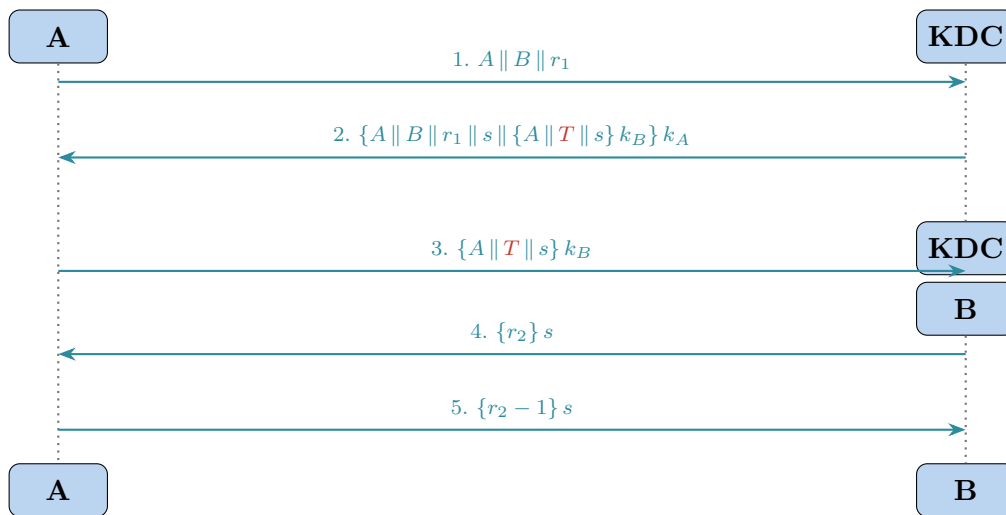
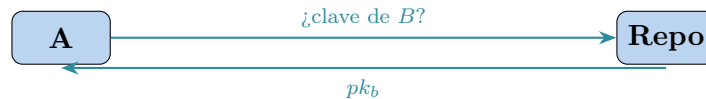


Figura 5: Modificación Denning–Sacco: el timestamp T (en rojo) viaja dentro del ticket. (Recreación de la diapositiva.)

Ataques activos

Un atacante activo puede **omitir**, **reescribir**, **reordenar** o **repetir** mensajes. Los esquemas de intercambio ingenuos *no sirven* contra atacantes activos $\Rightarrow$ ataques **Man-in-the-Middle (MITM)**.

Lo que A imagina:



Lo que pasa en realidad:

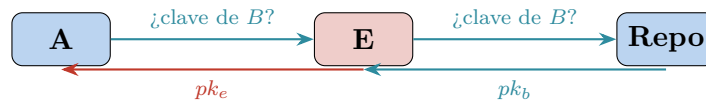


Figura 6: E se interpone entre A y el repositorio y entrega su clave pk_e en lugar de pk_b . (Recreación de las diapositivas “Problemas” y “Man in the middle”.)

Una vez que A tiene la clave equivocada, todo lo que cifre “para B ” lo abre E :

$$\underbrace{A \xrightarrow{e_{pk_e}(M)} E}_{E \text{ descifra}} \xrightarrow{e_{pk_b}(M)} B.$$

El problema entra en la esfera de la administración de claves: usar la clave equivocada significa que *no hay garantías de confidencialidad ni integridad*.

5 Infraestructura de Clave Pública (PKI)

Objetivo de la PKI

Asociar una identidad a las claves públicas para evitar la suplantación (*man-in-the-middle* / *spoofing*). **No es aplicable a criptosistemas simétricos.** *Motivo:* la selección de claves depende de las partes; usar la clave equivocada anula la confidencialidad y la integridad.

5.1 Certificados

Certificado

Es un mensaje que contiene **al menos:** información de identidad (p. ej. nombre), **clave pública asociada**, fecha de emisión, intervalo de validez y **tipo de uso** de la clave (firma de mensajes, cifrado de e-mails, firma de certificados, cifrado de sitios web). El certificado está, a su vez, **firmado digitalmente por una autoridad competente (CA)**.

Uso de un certificado. A solicita el certificado de B (puede proveerlo B o cualquier otra entidad). Con él, A puede:

- **Verificar la identidad** comparando el **CN** (*Common Name*): e-mails $\rightarrow$ CN=dirección; servidores $\rightarrow$ CN=IP o *hostname*; empresas $\rightarrow$ CN=razón social.
- **Obtener la clave pública de B :** es parte del propio certificado e incluye el tipo de clave (p. ej. RSA-2048, DSA-EC 320).
- **Verificar la validez:** tipo de uso permitido y fecha de vigencia.
- **Verificar la integridad:** validando la **firma digital** de la autoridad.

El gran “pero”: la validación de una firma digital *requiere una clave pública*. ¿Cómo se obtiene la clave pública *de la autoridad*? $\Rightarrow$ cadenas de firmas.

5.2 Cadenas de firmas y CA raíces

Cadenas de certificados

Las autoridades certificadoras tienen, a su vez, un certificado. Se acostumbra incluir el certificado de la autoridad junto con cada certificado. ¿Quién valida el certificado de la CA? **Otra CA.**

- **CA raíces:** firman su *propio* certificado (auto-firmado). Son el **punto de confianza** del sistema.
- **Concepto de raíz:** confiar en una única CA que *delega* en otras la capacidad de firmar certificados.

Ejemplos (la cadena se concatena $\parallel$ del sujeto hacia la raíz):

Cert. de A firmado por CA3: $C_a = C'_a \parallel C_{CA3} \parallel C_{CA2} \parallel C_{CA1}$

Cert. de B firmado por CA1: $C_b = C'_b \parallel C_{CA1}$

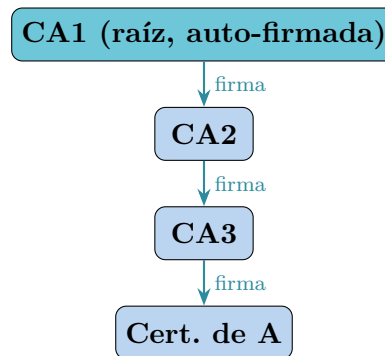


Figura 7: Cadena de confianza: la raíz delega y firma hacia abajo hasta el certificado final.

Validación entre CA distintas

No existe una única CA raíz. Si *A* y *B* tienen CA diferentes, ¿cómo valida *A* el certificado de *B*?

- **Opción 1:** *A* confía en la CA de *B*.
- **Opción 2:** las CA se certifican entre sí (cada una emite un certificado de la identidad de la otra).

En la práctica: existe una **lista de CA reconocidas** embebida en el sistema operativo, en los navegadores y en *runtimes* como la JVM.

5.3 Certificados X.509

X.509 — estándar de certificados digitales

Incluye: **versión**, número de serie, identificador del algoritmo de firma, nombre del emisor (CA), intervalo de validez, nombre del sujeto (CN), **clave pública del sujeto y firma** (firma del hash de todo lo anterior). Forman cadenas de certificados.

Ejemplo (recorte de un X.509 real, formato OpenSSL):

```

Version: 3 (0x2)
Serial Number: 1 (0x1)
Signature Algorithm: md5WithRSAEncryption
Issuer: C=ZA, ST=Western Cape, ... CN=Thawte Server CA/Email=...
Validity
  Not Before: Aug 1 00:00:00 1996 GMT
  Not After : Dec 31 23:59:59 2020 GMT
Subject: C=ZA, ... CN=Thawte Server CA/Email=...
Subject Public Key Info:
  Public Key Algorithm: rsaEncryption
  RSA Public Key: (1024 bit) ... Exponent: 65537 (0x10001)
X509v3 extensions:
  X509v3 Basic Constraints: critical -> CA:TRUE
Signature Algorithm: md5WithRSAEncryption 07:fa:4c:69:...
    
```

Verificación de un certificado X.509

1. **Obtener la clave pública del emisor:** de la cadena de certificados, o del sistema si es raíz. Si hace falta, verificar *recursivamente* el certificado del emisor.
2. **Verificar la integridad:** con el algoritmo especificado y la clave pública del emisor (validar la firma).
3. **Verificar el intervalo de validez:** debe estar vigente; la CA debe haber estado vigente al *comienzo* del período de vigencia.
4. **Verificar la identidad:** comparar el CN con el que espera la aplicación (según el uso del certificado).
5. **Verificar el uso:** que el certificado esté autorizado para el uso pretendido.

Ejercicio (práctica/teoría) — cadena www.itba.edu.ar

La diapositiva muestra una cadena real de 4 certificados. **Las validaciones necesarias** para aceptar el del servidor:

1. **Servidor** (CN www.itba.edu.ar, SAN www.itba.edu.ar, emisor *Amazon*): chequear que el CN/SAN coincide con el host pretendido; vigencia (6/6/2017–7/7/2018); validar su firma con la clave pública de *Amazon*; uso permitido para TLS de servidor.
2. **Intermedio** (CN *Amazon*, emisor *Amazon Root CA 1*): validar su firma con la clave de *Amazon Root CA 1*; vigencia; que tenga atributo de CA.
3. **Intermedio/raíz** (CN *Amazon Root CA 1*, emisor *Starfield Services Root CA - G2*): validar su firma con la clave de *Starfield*; vigencia.
4. **Raíz** (CN *Starfield Services Root CA - G2*, **auto-firmada**): debe estar en la **lista de CA confiables** del SO/navegador; vigencia (1/9/2009–28/6/2034).

En cada eslabón: la clave pública del *emisor* valida la firma del *sujeto*, hasta llegar a una raíz confiada. Además, conviene chequear que ninguno esté **revocado**.

5.4 Revocación: CRL y OCSP

Revocación de claves

Necesidad de invalidar una clave **antes** de su expiración: fue averiguada por un atacante, o hay un cambio anticipado (cambio de dueño). *Problemas*: no revocar claves que no deben revocarse (evitar revocaciones sin autorización) y **propagar** la información para evitar futuras comunicaciones con esa clave.

- **CRL (Listas de revocación)**: listas de certificados revocados, análogas a las listas de tarjetas de crédito robadas. *Actual*: vigentes y revocados; *histórico*: expirados y revocados. En X.509 **solo el emisor de un certificado puede revocarlo**; se agrega al CRL de la CA y puede validarse *offline* u *online*.
- **OCSP (Online Certificate Status Protocol)**: certifica que una firma está “activa”. Servicio en línea con API estandarizada, administrado por las CA; el resultado está **firmado** (es un certificado de *corta duración*: 1 hora a 7 días). **OCSP Stapling**: el resultado se incluye en la cadena, de modo que el cliente no consulta una fuente externa (recomendado para sitios de alto tráfico).

Resumen de certificados: asocian una clave pública con una identidad; se pueden validar *offline* (¡cuidado con las revocaciones!); están certificados por otra entidad; forman cadenas de confianza; **resuelven la integridad de las claves públicas**.

6 Canal seguro: SSL / TLS

Canal seguro

Confidencialidad e integridad sobre un canal inseguro. SSL/TLS ofrece seguridad a **nivel transporte**, sobre un canal confiable (TCP). Provee **confidencialidad**, **integridad** y **autenticación** de los participantes (origen y destino).

- **SSL** (Secure Socket Layer): versión inicial, creada por Netscape.
- **TLS** (Transport Layer Security): evolución de SSL (TLS 1.2 $\equiv$ SSL 3.3), mejora deficiencias; es el **estándar actual**. Depende de la **PKI**.

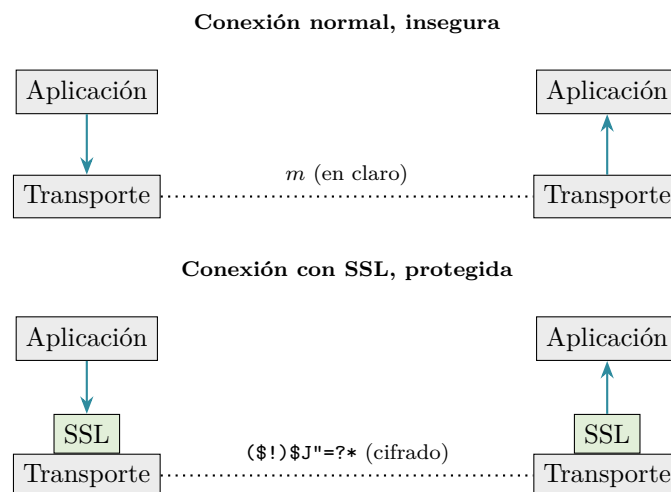


Figura 8: SSL se inserta entre aplicación y transporte. (Recreación de la diapositiva “Concepto”.)

6.1 TLS Record

TLS recibe un mensaje a enviar, lo **divide en bloques** de no más de 2^{16} bytes, **comprime** cada bloque y **calcula su hash**, **cifra** cada bloque y su hash, y lo **envía** al servicio inferior (TCP). Cada *record* lleva un **header**.

6.2 Tipos de mensajes y material criptográfico

TLS handshake	Configuración inicial, autenticación y negociación del material criptográfico.
TLS application data	Información del protocolo encapsulado.
TLS alert	Informa eventos y problemas (advertencia o fatal).
TLS change cipher spec	Concluye una renegociación de claves.

Criptografía soportada (lo vigente en azul en las slides):

- *Intercambio de clave*: **RSA**, **DH con certificado RSA o DSS**, DH anónimo, **ECDH** (DH sobre curvas elípticas), Kerberos. (Variantes obsoletas de 512 bits por restricciones de exportación de EE.UU.)
- *Cifrado simétrico*: ChaCha20, 3DES-EDE-CBC, **AES** (CBC, CCM, **GCM**), IDEA-CBC, ARIA. (Obsoletos: RC4, DES-CBC, claves de 40 bits.)
- *Hash/MAC*: HMAC-MD5, HMAC-SHA1, **HMAC-SHA2 (256/384)**, **AEAD** (Authenticated Encryption with Associated Data).
- *Autenticación*: firmas RSA, **firmas DSS**.

Sesión vs. conexión TLS

Sesión: asociación entre dos pares; puede soportar múltiples conexiones. Contiene: identificador único, certificado X.509v3 del otro extremo (opcional), método de compresión, método de cifrado y MAC acordados, y el **Master Secret** (clave de sesión compartida de 48 bytes).

Conexión: describe cómo intercambiar datos. Contiene: secuencia de bits aleatoria de calidad criptográfica, claves de escritura (distintas) para cada lado, claves para los MAC, IVs si hacen falta, y números de secuencia para cliente y servidor.

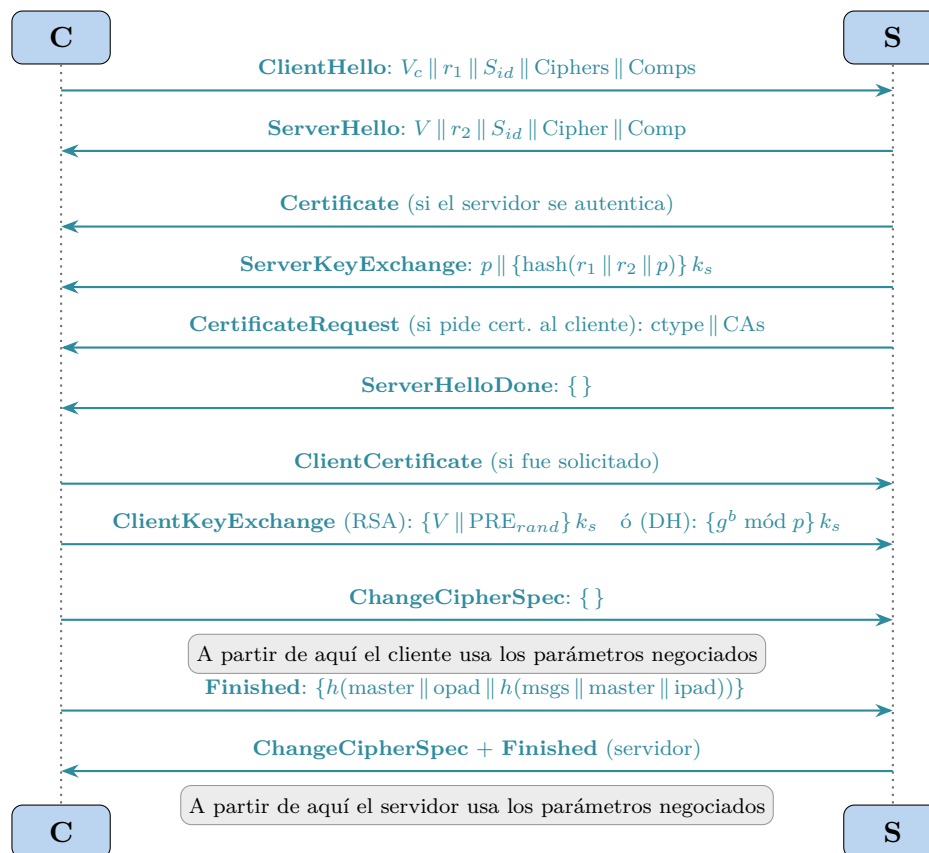
6.3 TLS Handshake (diagrama de secuencia)


Figura 9: Handshake completo de TLS. (Recreación de las 4 partes de las diapositivas.)

Parámetros del handshake

- V_c : versión del cliente; $V = \min(V_c, V_{\text{servidor}})$.
- r_1, r_2 : **nonces** (timestamp + 28 bytes aleatorios); **evitan replay**.
- S_{id} : ID de sesión (0 para iniciar una nueva).
- Ciphers/Cipher, Comps/Comp: *stacks* criptográficos y métodos de compresión ofrecidos vs. el elegido.
- p : parámetros criptográficos (e, n para RSA; p, g, g^a para DH).
- k_s : clave privada del servidor (correspondiente a la pública del certificado).
- **Anti-downgrade**: el cliente reenvía la versión V originalmente informada dentro del ClientKeyExchange para **prevenir downgrade attacks**.
- DH: $\text{PRE} = g^{ab} \bmod p$ es el *pre-master secret*.

El **Master Secret** se deriva combinando pre, r_1, r_2 con MD5/SHA:

$$\text{Master} = \text{MD5}(\text{pre} \parallel \text{SHA}(A' \parallel \text{pre} \parallel r_1 \parallel r_2)) \parallel \text{MD5}(\text{pre} \parallel \text{SHA}(BB' \parallel \dots)) \parallel \dots$$

- **Change Cipher Spec**: usado durante el handshake, puede aparecer en cualquier momento; implica **renegociación** de las claves de sesión; lo solicita cliente o servidor; *no tiene contenido* (es un tipo de mensaje).
- **Alert**: eventos fuera de banda, de *advertencia* o *fatales* (un fatal invalida la conexión). **CloseNotify** indica el fin de una sesión. Errores fatales: mensaje no esperado, MAC incorrecto, error al descomprimir, error de handshake, parámetro ilegal. Advertencias: no hay certificado / no válido / no soportado / expirado / revocado.

Panorama: TLS es el estándar de comunicación segura en servicios web (HTTPS); depende de la PKI; no suele usarse para validar clientes; soportado por todos los navegadores (con distinta respuesta frente a alertas). Problemas de diseño en los clientes le hicieron perder parte de su utilidad, pero *sigue siendo la alternativa más adoptada*.

7 Criptografía de umbrales (Threshold Cryptography)

Criptosistema de umbrales

Es una terna de algoritmos:

$$\begin{aligned} \text{Gen} : () &\rightarrow K^N && \text{(generador de clave)} \\ \text{Enc} : K^T \times P &\rightarrow C && \text{(cifrado)} \\ \text{Dec} : K^T \times C &\rightarrow P && \text{(descifrado)} \end{aligned}$$

Propiedad básica: para todo m y k^n válidos, para todo $k' = \{k^n\}_t$ y $k'' = \{k^n\}_t$ se cumple $d_{k''}(e_{k'}(m)) = m$.

Conceptos nuevos:

- N = **sombras** o cantidad de claves disponibles.
- T = **umbral**: cantidad de claves *necesarias*.
- K^X : X claves, cada una perteneciente a K ; $\{K^x\}_n$: subconjunto de n claves cualesquiera de K^X .

La idea: *cualquier* subconjunto de T de las N sombras alcanza para operar, pero con menos de T no se puede.

8 Secreto compartido de Shamir

Método de Shamir — esquema (t, n) threshold

Principio: un polinomio de grado $t - 1$ queda **unívocamente determinado por su evaluación en t puntos distintos**. El **secreto es el polinomio** (en particular, $s = P(0)$, el término independiente) y los **puntos son las sombras**.

Construcción (reparto): sea s el secreto a compartir.

- Se fija $a_0 = s$ y se eligen $a_1, \dots, a_{t-1} \leftarrow \{0, \dots, p-1\}$ al azar.
- Se arma $P(x) = a_{t-1}x^{t-1} + \dots + a_1x + a_0$ (mód p), con $p > s$ y $p > n$ (p primo).
- Las sombras son $P(1), P(2), \dots, P(n)$ (se pueden generar muchas: n a elección).

Reconstrucción: con t sombras cualesquiera $(s_{i_1}, \dots, s_{i_t})$, $s_{i_a} = P(i_a)$, se interpola por Lagrange:

$$P(x) = \sum_{a=1}^t \left(s_{i_a} \prod_{\substack{b=1 \\ b \neq a}}^t \frac{x - i_b}{i_a - i_b} \right)$$

y se evalúa $s = P(0)$.

8.1 Intuición geométrica

Aporte de la práctica — la recta, la parábola y las imágenes

La práctica desarrolla la intuición:

- **Grado 1 (recta) $\Rightarrow$ esquema $(2, n)$:** hacen falta 2 puntos para trazar una recta. Con un solo punto pasan **infinitas** rectas $\Rightarrow$ secreto indeterminado.
- **Grado 2 (parábola) $\Rightarrow$ esquema $(3, n)$:** hacen falta 3 puntos. Con 1 o 2 puntos, hay miles de parábolas posibles.
- En general: **número de sombras necesarias = grado del polinomio + 1 = t** . Se pueden repartir *muchas* sombras (a todo un curso), pero juntando cualesquiera t se recupera el secreto, y con menos de t *no se sabe nada*.
- **Secreto compartido por imágenes:** una imagen secreta se divide en “capas” que se reparten como imágenes; superponiendo t de ellas se reconstruye la imagen original (misma idea (t, n) aplicada a píxeles).

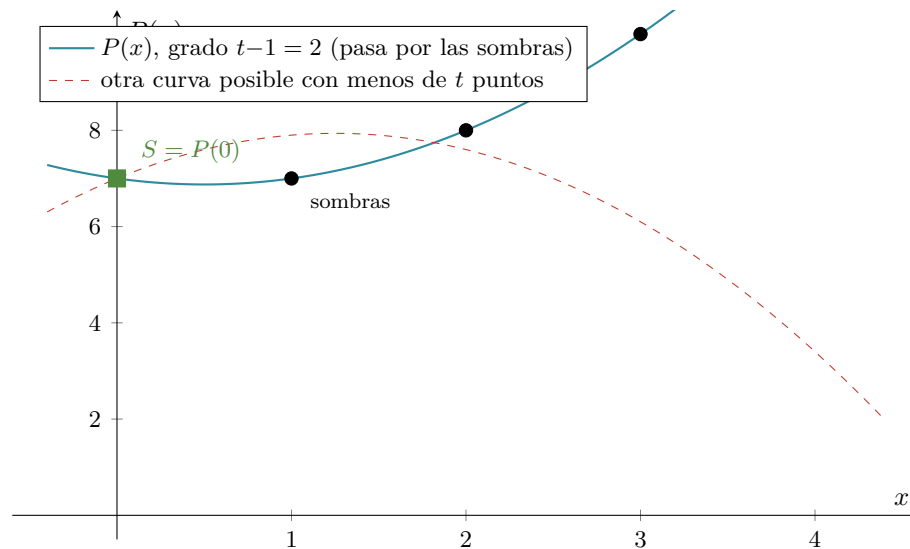


Figura 10: Secreto compartido de Shamir (esquema conceptual): el polinomio de grado $t - 1$ pasa por las sombras (puntos negros); el secreto es $S = P(0)$ (cuadrado verde). Con *menos* de t puntos hay **infinitas** curvas posibles (la curva roja punteada ilustra una alternativa que pasa por el mismo S).

8.2 Ejemplo de cifrado (reparto) — esquema (3, 5)

Ejemplo resuelto (diapositiva “Ejemplo Cifrado”)

Secreto $s = 7$, esquema (3, 5), en $\mathbb{Z}_{11}$. Se elige el polinomio de grado $t - 1 = 2$:

$$P(x) = 5x^2 + 3x + 7 \quad (\text{mód } 11)$$

Se calculan las sombras $P(1), \dots, P(5)$:

$$P(1) = 5 + 3 + 7 = 15 \equiv 4 \quad (\text{mód } 11)$$

$$P(2) = 20 + 6 + 7 = 33 \equiv 0 \quad (\text{mód } 11)$$

$$P(3) = 45 + 9 + 7 = 61 \equiv 6 \quad (\text{mód } 11)$$

$$P(4) = 80 + 12 + 7 = 99 \equiv \mathbf{0} \quad (\text{mód } 11)$$

$$P(5) = 125 + 15 + 7 = 147 \equiv 4 \quad (\text{mód } 11)$$

Sombras: (1, 4), (2, 0), (3, 6), (4, 0), (5, 4).

Errata de la diapositiva — $P(4)$

La diapositiva original escribe $P(4) = 2$, pero $80 + 12 + 7 = 99 = 9 \cdot 11$, luego $P(4) \equiv \mathbf{0}$ (mód 11), *no* 2. Es un error aritmético de la slide. La reconstrucción de la slide usa (2, 0), (3, 6), (5, 4) —que *no* incluyen la sombra 4—, por lo que el resultado $s = 7$ sigue siendo correcto. (Verificado aparte: las sombras correctas son 4, 0, 6, 0, 4.)

8.3 Ejemplo de descifrado (reconstrucción por Lagrange)

Ejemplo resuelto (diapositiva “Ejemplo Descifrado”)

Tomando tres sombras cualesquiera, p. ej. $(2, 0)$, $(3, 6)$, $(5, 4)$, se interpola por Lagrange (todo mód 11):

$$P(x) = \sum_{a=1}^t \left(s_{i_a} \prod_{\substack{b=1 \\ b \neq a}}^t \frac{x - i_b}{i_a - i_b} \right)$$

$$P(x) = \left[0 \cdot \frac{(x-3)(x-5)}{(2-3)(2-5)} + 6 \cdot \frac{(x-2)(x-5)}{(3-2)(3-5)} + 4 \cdot \frac{(x-2)(x-3)}{(5-2)(5-3)} \right]$$

$$P(x) = [0 - 3(x^2 - 7x + 10) + (4/6)(x^2 - 5x + 6)] \text{ mód } 11$$

$$P(x) = 5x^2 + 3x + 7 \quad \implies \quad \boxed{s = P(0) = 7}$$

Recordando que en $\mathbb{Z}_{11}$ la división es multiplicación por el inverso: el término medio $\frac{6}{(1)(-2)} = \frac{6}{-2}$ y $4/6$ se resuelven con la tabla de inversos modulares.

Aporte de la práctica — aritmética modular e implementación

La práctica desarrolla un *segundo* ejemplo equivalente: $s = 7$, esquema $(3, n)$ con $P(x) = 5x^2 + 2x + 7$ (variante mostrada en clase) en $\mathbb{Z}_{11}$, resuelto de dos maneras:

- **Por sistema de ecuaciones (Gauss):** plantear $a_0 + a_1x + a_2x^2$ para cada sombra y resolver el sistema 3×3 con eliminación gaussiana *módulo 11* (pivoteo, restando filas). Cómodo en papel, incómodo en código (arrastra las x).
- **Por interpolación de Lagrange:** más fácil de *implementar* (no arrastra las x). Es la recomendada para el TP.

Claves de la aritmética modular (todo mód 11):

- *Dividir* por $d = \text{multiplicar por } d^{-1}$. Ej.: $10^{-1} = 10$ (pues $10 \cdot 10 = 100 \equiv 1$); $5^{-1} = 9$ (pues $5 \cdot 9 = 45 \equiv 1$); $8^{-1} = 7$ (pues $8 \cdot 7 = 56 \equiv 1$).
- Conviene tener a mano la **tabla de inversos modulares**.

En el TP se usa $\mathbb{Z}_{251}$ (el primo más grande que entra en un byte) en lugar de $\mathbb{Z}_{11}$.

9 Cross-check teoría vs. práctica

Tema	Teoría (slides)	Práctica (slides + transcript)
KDC	Mencionado como base de Needham-Schroeder	Desarrolla el flujo $A \rightarrow KDC \rightarrow A, B$, pros/contras, Kerberos
Needham-Schroeder	Las 3 aproximaciones + Denning-Sacco, paso a paso	Encuadre como limitación de clave privada
MITM / Masquerading	MITM sobre claves públicas	Distingue MITM (activo, ambos extremos) de Masquerading (un solo extremo); replay y key reuse
PKI / Certificados	X.509, cadenas, CRL/OCSP, verificación	Certificado vs. clave pública “pelada”; PKI Argentina
TLS	Handshake completo, record, alert	Resumen conf./aut./integridad; DH <i>autenticado</i> ; RFC 2246/8446
Shamir	Construcción + Lagrange + ejemplo (3, 5)	Intuición recta/parábola, imágenes, Gauss vs. Lagrange, inversos modulares, $\mathbb{Z}_{251}$ en TP

Lo que agrega la práctica: (1) el encuadre KDC como solución a la distribución/almacenamiento de claves; (2) el detalle del flujo del KDC con pros y contras; (3) la distinción precisa MITM vs. Masquerading y el rol de replay/key-reuse; (4) la intuición geométrica de Shamir (recta/parábola/imágenes); (5) la resolución *completa* del ejemplo de reconstrucción y la mecánica de la aritmética modular (inversos) con ambos métodos (Gauss y Lagrange).

Lectura recomendada

Capítulo 11 — *Computer Security: Art and Science* (Matt Bishop)

RFC 8446 — TLS v1.3 (<http://tools.ietf.org/html/rfc8446>)

RFC 2246 — TLS (primera versión)

Applied Cryptography — Bruce Schneier

Vulnerabilidad en la renegociación de claves:

<http://www.g-sec.lu/practicaltls.pdf>

Apunte integral de la Clase 5 — Protocolos · Criptografía y Seguridad, ITBA.

Síntesis de teoría (diapositivas + transcripción) y práctica (KDC, Needham-Schroeder, Denning-Sacco, TLS y Secreto Compartido de Shamir).